

The Personhood-Gate Handoff: Drawing the Automation Boundary at the False-Signal Line

Zain Dana Harper

Seattle, WA | [ORCID 0009-0001-7175-5393](https://orcid.org/0009-0001-7175-5393)

<https://github.com/HarperZ9>

July 2026

Abstract

A tool that drives a browser or a desktop for a person forces one boundary question. Which steps may the tool take for that person, and which must the person take in person? The usual framing asks whether a step is “automated” or “manual.” This note argues that this axis is wrong. It places the line on the person’s side of the interaction, where the line is fuzzy and moves under pressure. We propose a sharper test anchored at the counterparty, meaning the third party on the other side of the interaction. A step must hand control back to the operator exactly when it would transmit, to a third party, a false signal that a human is present. The operator here is the person the tool works for. Under this test the handoff points appear on their own. They are a CAPTCHA, a login, a multi-factor prompt, a legal attestation, an electronic signature, and a payment. Each one is a third party asking the operator to personally prove they are human, authenticate, attest, sign, or pay. We describe a small implementation. It detects these gates on a live page, hands off with a witnessed record of what was prepared, and enforces in code that no automated step submits past a gate. We also quarantine the inverse capability, meaning code that forges the human-present signal, behind an owned-host guard, so it stays unreachable from the operator-facing path. We are explicit that the research novelty is modest. This work overlaps attended automation and human-in-the-loop delegation. The contribution is the predicate and its enforcement. The boundary is checkable on any page and encoded as passing tests. Enforcement lives in code, so it does not depend on operator discipline.

In plain terms

Imagine you ask a helper to run an errand for you on the web. The errand might be renewing a permit or booking an appointment. Most of the steps are things the helper can do for you. It can find the right page and type your name and address into a form. Gathering the documents you will need is also its job. None of this work pretends the helper is you.

Now think about the steps that are different. The website may show a small test that asks “are you a real person?” Signing in to your account is another. Some sites send a code to your phone and wait for you to type it back. You might also be asked to swear that the facts you gave are true, to add your signature, or to pay. Each of these steps is the website asking *you*, in person, to prove something only you can prove. If the helper does them, it is telling the website that you are present when you are not.

So the errand splits in two. The helper can handle most steps for you. A few of them fall to you alone. The hard part is knowing where the split falls. The common way to decide is to ask how much you are willing to let the helper do. That question is about your comfort, and your comfort drifts. When you are busy, you let the helper do more. Step by step the helper ends up doing things it should not.

This note moves the question to a better place. The useful question is what the helper would tell the website. If a step would tell the website that a human is there when a machine is acting, the helper must stop and hand that step to you. This single question draws a clean line. It puts the login, the human-check, the code from your phone, the sworn statement, the signature, and the payment on your side. Everything else stays on the helper's side. The rest of this note states the question precisely, then shows how a tool detects these gates on a live page and refuses to cross the line even when a script author forgets to stop it.

1 The wrong axis

Plainly: asking whether a step is automated or manual puts the line on the operator's side, where the line is fuzzy and slips under pressure.

An agent that drives a browser for its operator reaches, again and again, a point where it could go on or could stop. The common way to reason about that point asks one thing. Is this step “automated,” or should it be “manual”? That question puts the boundary on the operator's side. It asks how comfortable the operator is with delegating, and it weighs how present and trusting the operator is that day. This axis is fuzzy and drifts under time pressure. An over-eager automation erodes exactly this axis, one convenient step at a time. It also answers the wrong question. When a website shows a CAPTCHA, meaning a test that checks whether a human or a program is acting, it poses a third party's question. That question is whether a human is here, now, for this action. The operator cannot answer it by proxy.

2 The predicate

Plainly: move the line to the other side, and hand control back to the operator whenever a step would tell a third party that a human is present when a machine is the one acting.

We propose moving the boundary to the counterparty with a single predicate, meaning a single yes-or-no test applied to each step:

Would this step transmit, to a third party, a signal asserting that a human is present, when the truth is that automation is acting?

If the answer is yes, the step is a *personhood gate*, and the tool hands off to the operator. A no lets the tool proceed. The predicate earns its place on three counts. It is *checkable*, meaning it can be evaluated against the concrete controls on a live page. Being *counterparty-anchored*, its result turns only on what the tool would tell someone else. The third property is that it *coincides with genuine operator presence*: if the operator is truly at the keyboard, crossing the gate costs them seconds. So the predicate is only expensive under unattended operation, which is the case it is designed to prevent.

The gates fall out of the predicate directly. A **CAPTCHA** is literally an “are you human” check. A **login** claims to be the account owner. An **MFA** prompt, meaning a multi-factor authentication prompt, demands a second, fresh factor. A **legal attestation**, such as “I certify, under penalty...”, asks the operator to personally assert a truth. An **e-signature** records a personal act. A **payment** authorizes money. Each one requests a human presence or a personal authority. Forging any of them transmits a falsehood to the counterparty, wherever the operator sits.

3 Implementation

Plainly: a small module reads the page, names the gate, does all the work up to it, stops, and the tool itself refuses to submit past a gate.

We implemented the predicate as a small module over a browser-automation tool. A pure classifier scans the current page for the signatures of each gate class. It returns the gate, or NONE, together with the operator action required. Its output is unit-tested against fixtures for each class. The classes carry an ordering, so the classifier names the most consequential personal act present. From most to least consequential, the order is payment, e-signature, legal attestation, CAPTCHA, MFA, and login. A *prepare* step does all the work up to a gate. It fills a form from the operator’s own profile, and it detects the terminal gate and the still-required fields. Then it stops without submitting. The system treats reaching a gate as a successful handoff and records a witnessed receipt of what was prepared and what the operator must do.

Two enforcement properties matter. First, the boundary is enforced by the runner itself, meaning the part of the tool that carries out the steps. Before any submit-class step, the runner re-checks for a personhood gate and halts if one is present. So a workflow cannot step past a gate even if its author forgot to insert a handoff. Second, the inverse capability stays in the codebase behind a wall. The inverse capability is code that forges the human-present signal. It patches the automation fingerprint, seeds behavioral signals, solves a CAPTCHA, and harvests a score token. This code lives behind a guard. The guard refuses to run unless the live page’s host is one the operator has explicitly named as owned. The operator-facing dispatcher and the workflow registry never import this code. Understanding how a bot-score gate works is legitimate defensive knowledge, and testing one’s own property is a real task. The wall keeps the capability available for those uses only. A test suite asserts that none of the forging actions is reachable from the shipped surface.

4 Scope and novelty

Plainly: the contribution is a clear test for where the line belongs, plus code that enforces it.

The predicate has a clean converse worth stating. An “operator in the driver’s seat” is only truly in the seat if they are the one who answers “are you human,” because they are. A tool that forges that answer removes the human at exactly the gate that asks for one. That defeats operator control. The personhood-gate handoff keeps that from happening.

We are candid about the research novelty, which is modest. Attended automation, human-in-the-loop delegation, and consent-gated actions are established. A password manager already fills a credential and defers a second factor. Detecting login or CAPTCHA widgets is routine. What we contribute is a crisp, counterparty-anchored predicate for where the boundary belongs, together with an enforcement of it. The boundary is evaluated on the page, checked before every submit in code, and backed by tests. Code and tests hold it in place. That shift is small. It moves the question from “how much do we automate” to “what would we tell a third party.” It also turns a fuzzy, erodable line into a checkable invariant.

5 Conclusion

Plainly: draw the line where the tool would misrepresent the operator to someone else, put it in code, and it holds.

The safest boundary for operator-driven automation is the line at which the tool would misrepresent the operator’s presence to someone else. Placed there, the handoff points are the login, the human-check, the attestation, the signature, and the payment. These are the acts that are irreducibly the operator’s, and the tool can do everything else. The line is worth drawing in code. A boundary the operator must keep in mind will slip as attention drifts. When the runner enforces it, the boundary holds.